

The Cost of Context: Modality-Aware Retrieval for LLM-Based Code Agents

GSV Prharsha

gsvprharsha.work@gmail.com

Abstract

Context window capacity imposes a practical limitation on LLM-based coding agents: tokens spent loading source code reduce the space available for reasoning, tool interaction, and response generation. For repositories of moderate to large scale, naive file loading quickly becomes inefficient and, in many cases, infeasible within current context limits. We present a modality-decomposed retrieval architecture that addresses this constraint by separating repository knowledge into five categories — structure, behavior, dependencies, state, and history — and assembling only the subset relevant to a given task. Retrieval is performed by a deterministic four-stage pipeline. During development, we identified and mitigated a retrieval failure mode, which we term *Structural Hub Collapse*, in which systems lacking semantic guidance disproportionately surface high-connectivity infrastructure modules rather than task-relevant code. In a controlled two-agent evaluation, the proposed system reduced assembled context size by 75.8 percent relative to selective file traversal performed by an unassisted agent, while producing functionally equivalent outputs.

1 Introduction

Large language models have made it practical to delegate substantial portions of software engineering work to automated agents. These agents read source code, reason about it, and produce edits, tests, or explanations in response to a stated task. The mechanism that makes this possible — loading relevant code into the context window — is also the mechanism that limits it. Every token occupied by source code is a token unavailable for reasoning, tool interaction, and response generation. The context window is a finite shared resource, and the way code is loaded into it determines how much of that resource is left for everything else.

For small codebases this tension is manageable. For repositories of large scale it becomes a material constraint. A naive agent working on a codebase of a few hundred files cannot load all relevant source into context without either exceeding the window or crowding out its own working space. The common alternative — selective file reading, where the agent chooses which files to open — reduces the raw token count but does not resolve the underlying problem. An agent that reads selectively still loads files, not knowledge: a single file typically contains far more information than any one task requires, and the portions relevant to the task are interleaved with portions that are not.

The observation motivating this work is that the mismatch between files and tasks is structural, not incidental. A bug fix in an authentication module requires knowing what that module *does*

and what state it *mutates*; it does not require its full implementation history or its position in the module hierarchy. A refactoring task requires the opposite: structure and history matter, behavior is secondary. Source code is conventionally organized by what it *is* — files and directories — but tasks are defined by what they *need*, which is a function of kind of knowledge, not of file boundaries.

Human memory exhibits a similar form of separation. Different categories of knowledge are not stored as a single retrievable unit, but as distributed representations associated with different cognitive functions. The hippocampus is widely understood not primarily as a storage site, but as an indexing mechanism that reconstructs relevant memory traces in response to a given context [3]. Critically, retrieval is selective: the brain does not replay all associated information about a domain in order to answer a localized question. This work adopts the same principle for repository-scale code retrieval by separating distinct categories of repository knowledge and reconstructing only the subset required for a given task.

Rather than loading files into context, the system introduces two primitives. A *code unit* is the smallest independently addressable software entity in a repository: a function, a class, or a module. A *modality* is a distinct category of knowledge about a code unit, each extracted from a separate source — structure from the AST, behavior from summarization, dependencies from import and call graphs, state from mutation analysis, and history from version control. Each code unit is decomposed into five modality shards at ingestion time and stored independently. At query time, a retrieval layer acts as a hippocampus: given a task description, it identifies which units are relevant and assembles only the modality shards that the task type requires, subject to a hard token budget. The result is a context payload that is smaller than naive file loading and designed to be more precisely targeted to the informational requirements of the task.

2 Background and Related Work

2.1 Retrieval-Augmented Generation

Retrieval-augmented generation is a family of approaches [1] that ground language model outputs in an external document collection by retrieving relevant documents at inference time and appending them to the prompt. In the standard formulation, a query is encoded as a dense vector, the collection is searched for nearest neighbors in embedding space, and the top results are concatenated into the context. This approach is well-suited to open-domain question answering, where documents are semantically self-contained and queries can be matched against them by surface similarity.

Code repositories are not semantically homogeneous in the way that document collections are. A function’s syntactic signature, its intended behavior, its callers, the state it modifies, and the rationale recorded in its commit history are distinct categories of knowledge that happen to be stored in the same file. Treating a source file as a single retrievable unit conflates these distinctions. A retrieval system that returns a file in response to a task query delivers all five categories at once, regardless of which the task actually requires. This is the core limitation that motivates a modality-separated approach.

2.2 Context Window Limits in LLM Agents

A natural response to context scarcity is to increase the context window. Advances in transformer architecture have progressively extended the token limits of available models, and techniques such as sparse attention [5] reduce the computational cost of processing longer sequences. However, a larger window does not resolve the selection problem. Research has shown that language model performance degrades for information positioned in the middle of long contexts [2]: models tend to attend more strongly to tokens near the beginning and end of the input, and task-relevant content that falls in the middle of a large context block is disproportionately underused.

For coding agents, this has a practical consequence. Even given a context window large enough to hold an entire codebase, the tokens consumed by source code displace tokens available for reasoning, tool interaction, and intermediate steps. The binding constraint is not absolute capacity but effective capacity — the portion of the window that remains usable for productive work after context loading. This paper treats context size not as a resource to be maximized but as a cost to be minimized.

2.3 Hippocampal Memory and Distributed Knowledge Representation

The hippocampus is understood in cognitive neuroscience not as a general-purpose memory store but as an indexing structure [3]: it binds distributed cortical representations into retrievable episodes and, when retrieval is triggered, reconstructs a coherent representation from the relevant traces. Under the complementary learning systems framework [4], the hippocampus handles specific, rapidly acquired associations, while the neocortex encodes general structured knowledge across longer timescales. Different categories of knowledge — procedural, semantic, episodic — are encoded through distinct mechanisms and retrieved selectively in response to context demands.

The consequence relevant to this work is that retrieval is not a lookup of a stored record but a context-conditioned reconstruction from distributed, modality-separated storage. The brain does not load everything it knows about a domain before answering a question about part of it. This paper applies the same principle at the level of a code repository: five knowledge modalities are stored separately at ingestion time and assembled selectively at query time, with the task description determining which modalities are included.

2.4 Prior Agent and IDE Context Strategies

LLM-based software engineering agents have addressed context assembly as a behavioral problem: the agent is given file system tools and decides at runtime which files to open. SWE-agent [7] is a representative example, equipping the model with shell and editor commands that allow it to navigate and read a repository interactively. This approach avoids loading the entire codebase upfront, but it still loads full files — the agent reads files, not knowledge categories — and each file read delivers all knowledge types regardless of which the task requires. The agent must also decide what to read before having read it, which means retrieval decisions are made without the information they depend on.

Commercially deployed AI coding tools take a different approach. Modern incarnations extend the basic embedding pipeline with reranking, symbol indexing, and usage-pattern heuristics to improve retrieval precision. Editor integrations such as Cursor [9] and Windsurf [10], and workspace-aware

assistants such as GitHub Copilot [8], nonetheless share a common foundation: repository-level retrieval using embedding-based chunking. The general approach is to split source files into fixed-size overlapping token windows, embed each chunk using a code-specialized model, store the resulting vectors in an index, and at query time retrieve the top- k chunks most similar to the embedded query. The retrieved chunks are concatenated into the prompt alongside the current editor context.

This approach has two structural limitations relevant to the present work. First, chunking by token count does not respect semantic boundaries: a fixed-size window frequently splits a function midway, separates a class from its methods, or interleaves unrelated code from adjacent regions of a file. The retrieved chunk is a positional artifact rather than a coherent unit of knowledge. Second, all knowledge categories are mixed within each chunk. A chunk retrieved for a bug-fix task carries the function’s full source — signature, implementation, inline comments, and neighboring code — regardless of which parts are relevant to the fix. There is no mechanism to retrieve behavior without structure, or dependencies without implementation. The system described in this paper addresses both limitations by decomposing code at semantic unit boundaries and separating storage by knowledge type before any retrieval takes place.

3 System Architecture

The system is organized into two phases that execute at different times. The ingestion phase runs once per repository and decomposes every code unit into a set of modality-separated shards, which it persists to a structured store. The retrieval phase runs once per task, selecting and assembling the minimal shard subset required for that task subject to a hard token budget. Figure ?? shows the full system.

3.1 Ingestion Pipeline

The ingestion layer formalizes two primitives introduced in Section 1. A *code unit* is the smallest independently addressable entity in a Python repository: a module, a class, or a function or method. A unit is identified by a stable dot-separated string of the form `<module>.<class>.<function>` (e.g., `auth.middleware.check_token`). A *modality* is a distinct category of knowledge about a code unit, derived from a separate source and stored independently. The system defines five modalities: *structure* (derived from the AST), *behavior* (generated by a language model), *dependencies* (derived from import and call graph analysis), *state* (derived from AST mutation analysis), and *history* (derived from version control logs). A *shard* is a single (unit, modality) pair: a bounded text representation of one knowledge category for one code unit.

Ingestion begins with a decomposer (`decomposer.py`) that traverses every `.py` file in the target directory. For each file it parses the AST and extracts a list of `CodeUnit` objects: one module-level unit, one unit per top-level class, and one unit per method or function within each class or at module scope. Five parsers then run independently per unit, each producing a single `Shard`:

- **structure.py** uses the `ast` module to extract the unit’s syntactic signature and position in the class hierarchy: function signatures, class inheritance chains, decorator lists, and the presence of a docstring. It answers the question *what shape is this?* Confidence: 0.95.
- **dependencies.py** uses the `ast` module to extract import statements and intra-repository

call sites. In addition to producing a shard, it returns an edge list of tuples (`caller_id`, `callee_name`, "CALLS") consumed by the store to build the networkx DiGraph. It answers the question *what talks to what?* Confidence: 0.95.

- **state.py** extracts mutation-relevant information via the AST: `self.*` attribute assignments, module-level variable declarations, `global` and `nonlocal` declarations, and return-expression analysis. It answers the question *what does this change?* Confidence: 0.90.
- **history.py** queries `git log --follow` for the ten most recent commits touching the source file containing the unit, capturing commit messages and authorship metadata. If the repository carries no git history, the parser returns an empty shard. It answers the question *why is it this way?* Confidence: 0.70.
- **behavior.py** issues a single API call to a language model requesting a concise intent summary of the unit: what it does, what it assumes, and what side effects it produces. This is the only modality that involves an LLM call, and it is the only call made per unit during ingestion. If the API key is absent the parser returns `None` and the shard is omitted. It answers the question *what does this do?* Confidence: 0.80.

When sources conflict, the system resolves the disagreement using a confidence hierarchy. Structure and dependencies (0.95) take precedence over state (0.90), behavior (0.80), and history (0.70). Conflicts are logged with both sources and the winning value; no conflict is resolved silently. All token counts use the `tiktoken cl100k_base` encoding and are computed at parse time.

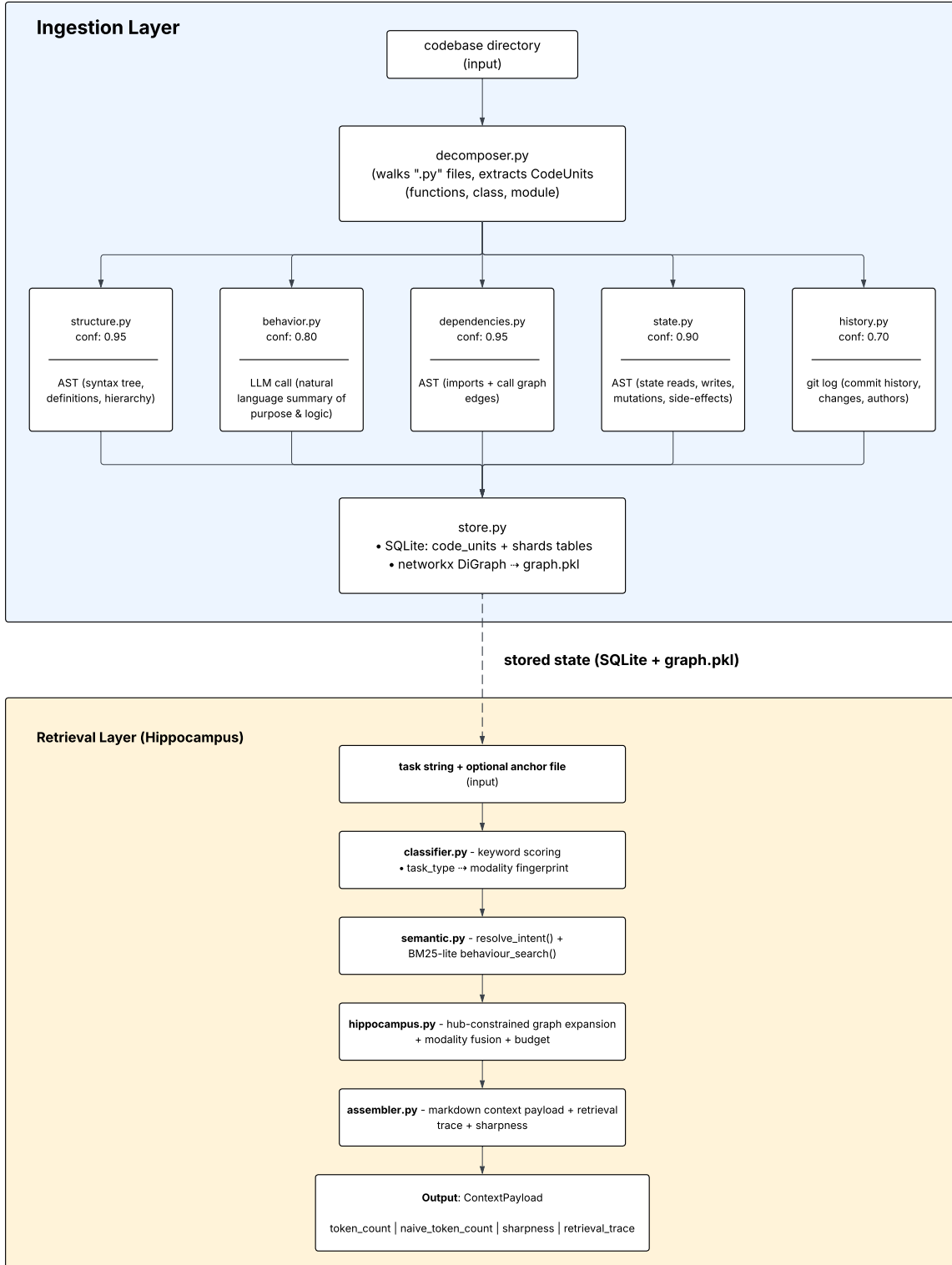


Figure 1: System architecture showing ingestion-time modality decomposition and retrieval-time hippocampal orchestration.

3.2 Shard Storage

Shards are persisted to a SQLite database using the `sqlite3` standard library module. The schema comprises two tables.

code_units stores one row per code unit:

<code>id</code>	<code>TEXT</code>	<code>PRIMARY KEY</code>	<code>-- "<module>.<class>.<function>"</code>
<code>name</code>	<code>TEXT</code>		
<code>unit_type</code>	<code>TEXT</code>		<code>-- "function" "class" "module"</code>
<code>file_path</code>	<code>TEXT</code>		
<code>language</code>	<code>TEXT</code>		<code>-- "python"</code>
<code>source_token_count</code>	<code>INTEGER</code>		<code>-- raw file tokens (naive baseline)</code>

shards stores one row per (unit, modality) pair:

<code>id</code>	<code>TEXT</code>	<code>PRIMARY KEY</code>	<code>-- "<unit_id>::<modality>"</code>
<code>unit_id</code>	<code>TEXT</code>		<code>-- FK -> code_units.id</code>
<code>modality</code>	<code>TEXT</code>		<code>-- "structure" "behavior" ...</code>
<code>content</code>	<code>TEXT</code>		
<code>token_count</code>	<code>INTEGER</code>		
<code>confidence</code>	<code>REAL</code>		<code>-- 0.0 to 1.0</code>
<code>created_at</code>	<code>TEXT</code>		<code>-- ISO-8601</code>

Each shard is identified by the composite key

`<unit_id>::<modality>` (e.g., `auth.middleware.check_token::state`)

This convention makes shard identity deterministic, human-readable, and independent of any auto-incremented surrogate key.

Inter-unit relationships are stored in a networkx `DiGraph` serialized to disk as `graph.pkl`. Nodes are code unit identifier strings. Edges carry a `rel_type` attribute encoding one of three relationship kinds: `CALLS` (intra-repository function invocations), `IMPORTS` (module-level import dependencies), and `DEFINED_IN` (class-to-method containment). The graph is used at retrieval time for neighbor expansion and degree-based hub detection.

The `source_token_count` field on `code_units` records the raw token count of the full source file containing the module unit. Summing this field across all module-type units yields the naive baseline: the total tokens a full-context approach would consume to load the entire repository.

3.3 Retrieval Layer Overview

At query time the system receives a task string and an optional anchor file path and passes them through a four-stage pipeline that produces a `ContextPayload` bounded by a hard 8,000-token budget. Figure ?? shows the pipeline in detail.

The classifier (`classifier.py`) maps the task string to one of five task types — `bug_fix`, `feature_add`, `refactor`, `explain`, or `onboard` — through keyword frequency scoring with no LLM call. Each task type is associated with a fixed modality fingerprint encoding which modalities the task class requires: `bug_fix` uses behavior, state, and dependencies; `refactor` uses structure, dependencies,

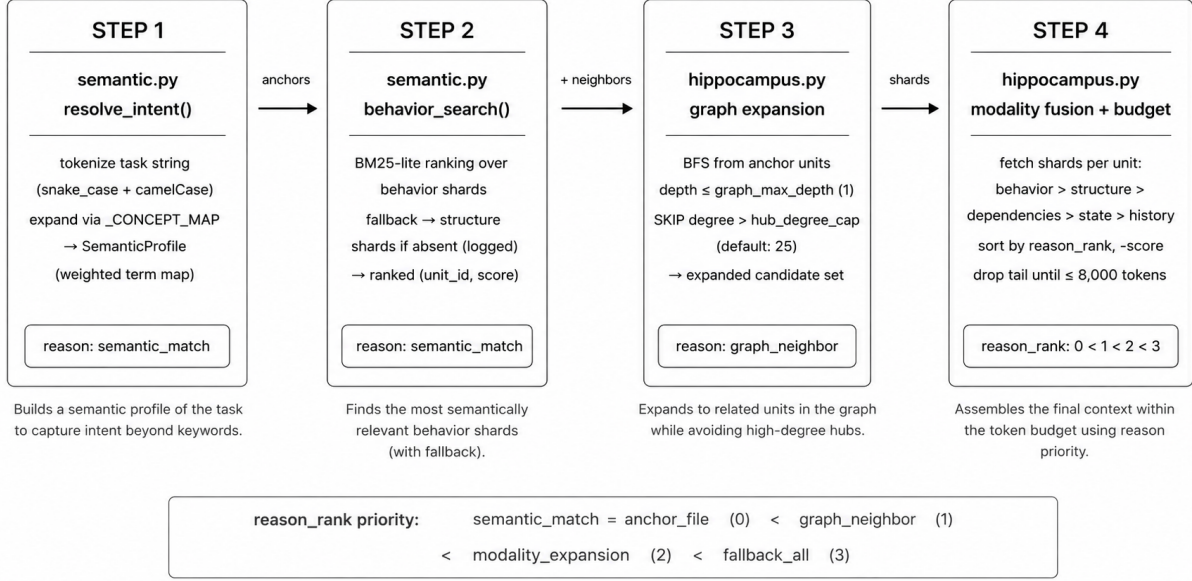


Figure 2: The four-stage retrieval pipeline. Each retrieved shard carries a retrieval reason that determines its priority during budget enforcement: semantically targeted shards survive; incidental modality-expansion shards are dropped first.

and history; **explain** uses behavior and structure. The fingerprint is applied in Stage 4 to determine which modalities are fetched per candidate unit.

Stages 1 and 2 implement semantic guidance, replacing the topology-biased seed-all-units approach of earlier designs. The intent resolver (**resolve_intent**) tokenizes the task string by splitting on snake_case underscores and camelCase boundaries, then expands task-domain terms through a static concept map: for example, the token *middleware* expands to {request, response, dispatch, pipeline, handler, intercept, filter}. Task tokens are weighted 2.0; concept-expansion tokens are weighted 1.0. The resulting **SemanticProfile** is a normalized weighted term-frequency map. The behavior search (**behavior_search**) then ranks all stored behavior shards using BM25-lite: for each shard, a relevance score is computed over the union of its unit identifier tokens and its content tokens against the **SemanticProfile**. The top **semantic_top_n** (default 10) units by score form the retrieval anchor set. When behavior shards are absent because ingestion ran without an API key, the search degrades to structure shards and emits a logged warning.

Stage 3 expands the anchor set by traversing the stored DiGraph. Starting from each anchor unit, a bounded breadth-first search to depth **graph_max_depth** (default 1) adds neighboring units to the candidate pool. The hub-degree cap is the critical constraint: any neighbor whose degree in the graph exceeds **hub_degree_cap** (default 25) is skipped unconditionally. High-degree nodes are structural hubs — application factory modules, centralized routing tables, and similar infrastructure files that are referenced by most of the codebase. Pulling them into the candidate set without this guard was the proximate cause of Structural Hub Collapse, described in detail in Section 5.

Stage 4 fetches shards for all candidate units in canonical modality order and applies the token budget. Shards are sorted by a four-key ordering: retrieval reason rank, descending BM25 score, descending confidence, and unit identifier (for full determinism). Reason rank encodes priority: semantically targeted shards (reason `semantic_match` or `anchor_file`, rank 0) survive before graph neighbors (rank 1), which survive before incidental modality-expansion shards (rank 2), which survive before fallback-all shards (rank 3). The enforcer removes shards from the tail of this ordering until the total token count is at or below the 8,000-token cap, logging each dropped shard identifier and its token cost.

The assembler (`assembler.py`) formats the surviving `RetrievedShard` objects into a structured markdown string, grouping shards by code unit and annotating each with its retrieval reason and score. It appends a retrieval trace section listing all selected shard identifiers. It also computes the retrieval sharpness metric, which is formally defined and analyzed in Section 5. The `ContextPayload` is the terminal output consumed by the calling agent.

4 Experimental Setup

The evaluation is structured as a controlled two-agent comparison across two open-source Python codebases. Each codebase was ingested once and queried with an identical set of five tasks, one per task type. This section describes the shared evaluation design, the ingestion conditions, and then each codebase in turn with its task set and results.

4.1 Evaluation Design

Two agents were given each task in isolation, with no knowledge of each other’s existence. We refer to the agent using the proposed modality-aware retrieval system as **MAR** (Modality-Aware Retrieval) and the unassisted baseline agent as **Naive** throughout this paper.

MAR received only the context payload assembled by the retrieval pipeline for that task. The payload was bounded at 8,000 tokens and contained modality-separated shards selected by the four-stage pipeline described in Section 3. MAR was instructed to answer using only the provided context.

Naive was given no pre-assembled context. It was provided with the full codebase directory tree and a set of file-reading tools, and navigated freely until it considered itself sufficiently informed to answer. The token count attributed to Naive is the sum of raw file tokens read during its traversal.

Both agents used the same model (claude-haiku-4-5-20251001), ran against the same codebase state, and received tasks in the same order. The primary measurement is assembled context size: the number of tokens each agent consumed before producing a response, used as a proxy for the context cost imposed on the downstream language model. A secondary measurement is retrieval sharpness for MAR, defined formally in Section 5.

4.2 Ingestion Conditions

Both codebases were ingested without an `ANTHROPIC_API_KEY` set at ingestion time. This means behavior shards were not generated: the `behavior.py` parser returned `None` for every unit and no behavior-modality rows were written to the shard store. All runs therefore reflect the proposed

system operating in *degraded mode*, in which the STEP 2 behavior search falls back to BM25-lite over structure shards rather than behavior shards. Retrieval remains deterministic and the hub-degree cap remains active, but semantic anchor quality is lower than it would be with full ingestion.

This condition was chosen deliberately. Generating behavior shards requires one LLM call per code unit, which at the scale of FastAPI represents a non-trivial inference cost. Degraded mode is the realistic deployment scenario for teams that have not yet conducted a full ingestion pass. Differences attributable to the presence or absence of behavior shards are analyzed in Section 7.

4.3 FastAPI

FastAPI is a modern asynchronous web framework for Python (version 0.115.x). The repository spans the framework source (`fastapi/`), an extensive documentation source tree (`docs_src/`), and a full test suite (`tests/`). The naive full-context baseline — the sum of raw token counts across all source files — is 689,221 tokens. FastAPI was chosen because its codebase exhibits the structural hub problem directly: `fastapi/routing.py` and `fastapi/applications.py` together account for over 54,000 raw tokens and are referenced by nearly every other module in the package, making them natural targets for hub collapse under naive retrieval.

Five tasks were defined, one per task type, written and frozen before any agent run.

ID	Type	Task description
T1	feature_add	Add request timing middleware that logs HTTP method, path, status code, and duration in milliseconds.
T2	bug_fix	Fix the issue where exception handlers do not catch exceptions raised inside background tasks.
T3	refactor	Refactor the path operation decorators (<code>get</code> , <code>post</code> , <code>put</code> , <code>delete</code> , <code>patch</code>) to eliminate repeated parameter definitions.
T4	explain	Explain how FastAPI validates and serializes response models, including <code>response_model_exclude_unset</code> and <code>response_model_exclude_none</code> .
T5	onboard	Overview of FastAPI’s dependency injection system: declaration, resolution, caching per request, and nesting.

Table 1: Task set for FastAPI (naive baseline: 689,221 tokens). Tasks were frozen before any agent run.

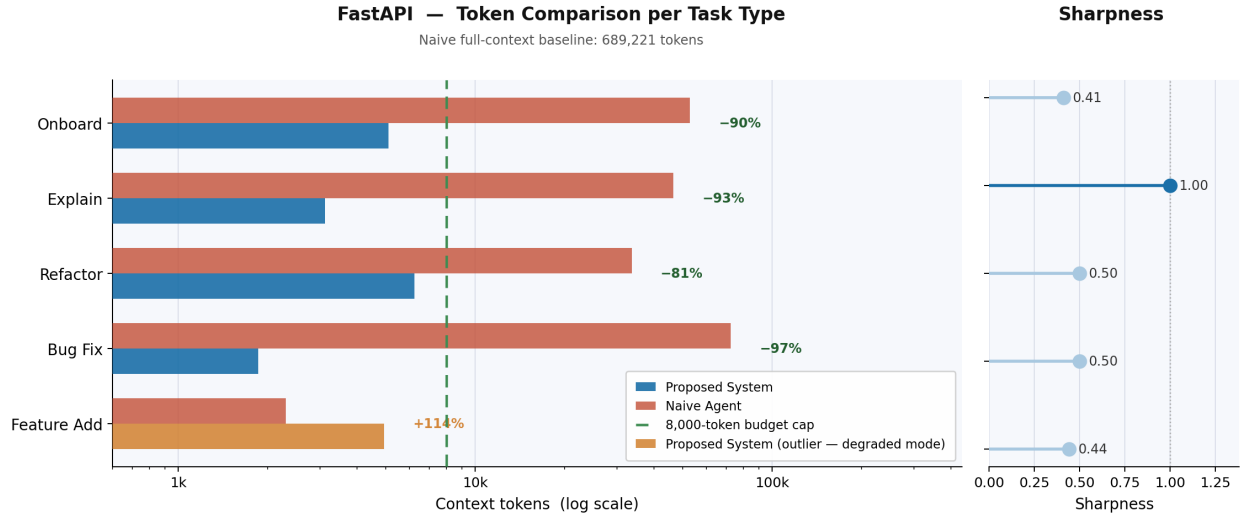


Figure 3: Token comparison per task type on FastAPI. MAR (blue) vs Naive (red) on a log scale; the dashed green line marks the 8,000-token budget cap. The amber bar for Feature Add is the T1 outlier: without behavior shards, the proposed system over-retrieved relative to Naive on this task. The sharpness panel (right) shows the fraction of selected shards that were semantically targeted.

4.4 Flask

Flask is a lightweight WSGI web framework (version 3.x). The naive full-context baseline is approximately 134,479 tokens. Flask was included as a contrast case: its smaller scope and flatter module structure represent a regime in which structural BM25 search should perform better without semantic guidance, providing a measurement of how the proposed system’s degraded-mode behavior scales with codebase complexity.

Five tasks were defined using the same taxonomy applied to FastAPI, written and frozen before any agent run.

ID	Type	Task description
T6	feature_add	Add a <code>before_request</code> hook that rate-limits by IP, returning HTTP 429.
T7	bug_fix	Fix <code>teardown_appcontext</code> callbacks not firing on unhandled exceptions.
T8	refactor	Refactor <code>add_url_rule</code> and the <code>route</code> decorator to share a single validation implementation.
T9	explain	Explain app context and request context: lifecycle, contents, and proxy resolution.
T10	onboard	Overview of Flask’s blueprint system: creation, registration, URL prefixes, and error handlers.

Table 2: Task set for Flask (naive baseline: 134,479 tokens). Tasks were frozen before any agent run.

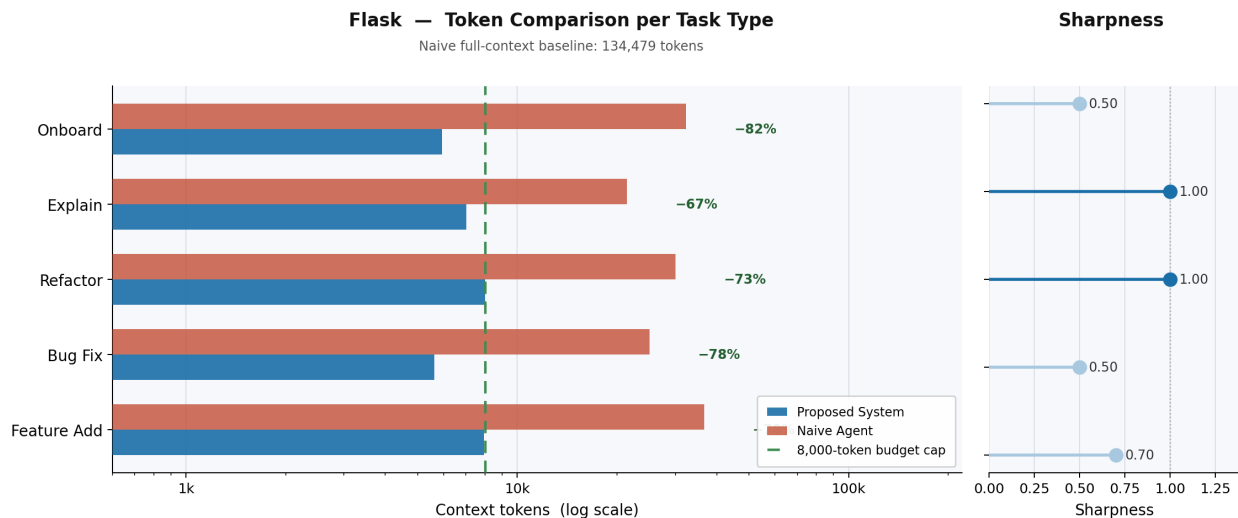


Figure 4: Token comparison per task type on Flask. Flask’s smaller codebase produces higher average sharpness (0.74 vs 0.57 for FastAPI); two tasks reach perfect sharpness (1.00), indicating that structure-based BM25 correctly targeted the relevant modules without behavior shards. All assembled payloads remain within the 8,000-token budget cap.

5 Structural Hub Collapse: Problem, Diagnosis, and Resolution

An early benchmark pilot of the retrieval system on the FastAPI codebase (run without an API key, so without behavior shards) produced a result that was technically impressive on one metric and entirely useless on another. Measured context size was reduced by 98.8 percent relative to the full naive baseline. The shards selected, however, bore no relation to the task at hand: for a query targeting exception-handling behavior, the assembled context was populated with tutorial files and high-connectivity infrastructure modules. The token count was right; the content was wrong.

This section names and explains that failure mode, defines the metric introduced to measure it, and describes the four changes that corrected it.

5.1 The Problem: Retrieval Collapses Toward Topological Hubs

The original retrieval pipeline seeded the candidate pool with all code units in the store, then performed one-hop graph expansion from every unit, and finally filled the 8,000-token budget by sorting candidates on confidence score alone. Under this scheme, every unit starts with identical priority, and the budget is decided entirely by the confidence values assigned to each modality: structure shards carry confidence 0.95, dependency shards 0.95, state shards 0.90, and history shards 0.70. Shard identity is irrelevant to the sort.

The failure follows directly. Python web frameworks have a power-law degree distribution in their dependency graphs. A small number of modules — routing tables, application factory classes, shared utility modules — are imported by nearly everything else in the codebase. These nodes have high in-degree. They are therefore reachable from almost any unit via one-hop graph expansion.

Once reached, their high-confidence structure shards enter the candidate pool and outrank shards from less-connected, more task-relevant files. The budget fills with infrastructure code.

This is *Structural Hub Collapse*: without semantic guidance, retrieval space collapses toward topological hubs. The assembled context is topologically representative of the codebase graph and semantically irrelevant to the task. On the FastAPI benchmark, `fastapi/routing.py` (25,701 tokens) and `fastapi/applications.py` (28,532 tokens) entered the retrieved set for tasks that had nothing to do with routing or application configuration. Naive’s independent file traversal confirmed the same failure from the naive side: it read `routing.py` in three of five tasks and `applications.py` in two of five, even for tasks whose answers resided in much smaller, more targeted files.

The analogy to biological memory is direct. A hippocampus that retrieved associations purely by activation frequency, rather than by contextual relevance, would surface the most-used memories — routine procedures, common facts — rather than the specific episodic trace needed for the current context. The original pipeline was doing exactly this at the level of the codebase.

5.2 Retrieval Sharpness

To make the failure mode measurable and the fix verifiable, a metric is introduced here and used consistently throughout Section 6.

Let T be the set of shards assembled for a given query, and let $T_{\text{targeted}} \subseteq T$ denote the subset whose retrieval reason is `semantic_match`, `anchor_file`, or `graph_neighbor` — that is, shards selected by a semantically guided mechanism rather than by topology or fallback. Retrieval sharpness is defined as:

$$\text{sharpness} = \frac{|T_{\text{targeted}}|}{|T|} \tag{1}$$

Sharpness of 1.0 indicates that every selected shard was intent-guided. Sharpness of 0.0 indicates that all shards entered via the `fallback_all` path, the hub-collapse regime. Each `RetrievedShard` object carries its retrieval reason, making sharpness computable without any additional instrumentation.

5.3 Resolution

Four coordinated changes shifted retrieval from topology-biased to intent-guided. All changes are deterministic and require no LLM calls during retrieval.

Semantic intent resolution. Before the store is queried, the task string is parsed into a weighted `SemanticProfile`. The string is tokenized on whitespace, snake_case underscores, and camelCase boundaries. Each token is then expanded through a static concept map: for example, the token `middleware` maps to `{request, response, dispatch, pipeline, handler}`. Task tokens are weighted 2.0; concept-expansion tokens 1.0. The resulting profile is a normalized term-frequency vector representing what the task is about, independent of any unit in the store.

Behavior-first BM25-lite search. Instead of seeding from all units, the pipeline ranks all stored behavior shards against the semantic profile using BM25-lite term frequency scoring and retains the

top- N units as anchors (default $N = 10$). These anchors receive retrieval reason `semantic_match`. When behavior shards are absent, as in the degraded-mode runs of Section 4, the search falls back to structure shards and logs a warning. This step replaces the seed-all approach: the candidate pool begins with units that are semantically related to the task rather than units that happen to be topologically central.

Hub-constrained graph expansion. A bounded breadth-first search from the semantic anchors adds neighboring units to the candidate pool, up to a configurable depth (default 1). Any neighbor whose degree in the dependency graph exceeds `hub_degree_cap` (default 25) is skipped unconditionally. This prevents expansion from flowing through high-degree infrastructure nodes. Included neighbors receive reason `graph_neighbor`. The hub-degree cap is the direct structural fix for hub collapse: nodes such as `routing.py` (degree 31) and `applications.py` (degree 28) cannot enter the candidate pool as collateral of graph traversal; they can only appear if they are themselves semantically matched.

Reason-aware budget enforcement. Shards are sorted by a four-key ordering: retrieval reason rank, descending BM25 score, descending confidence, and shard identifier for full determinism. Reason ranks encode priority: `semantic_match` and `anchor_file` carry rank 0, `graph_neighbor` rank 1, `modality_expansion` rank 2, and `fallback_all` rank 3. The budget enforcer drops shards from the tail of this ordering until the total token count is at or below 8,000. Under this scheme, a hub shard that enters via modality expansion is the first to be dropped when the budget is tight; a task-targeted shard survives the cut regardless of its modality confidence score.

5.4 Concrete Illustration: FastAPI Bug-Fix Task

The T2 task from Section 4 — *fix the issue where exception handlers do not catch exceptions raised inside background tasks* — provides a concrete trace of hub collapse and its resolution, because the relevant code is contained in two small modules (`fastapi/exception_handlers.py` and `fastapi/background.py`) while the two dominant hubs (`fastapi/routing.py`, 25,701 tokens; `fastapi/applications.py`, 28,532 tokens) are their high-degree graph neighbors.

Pre-fix behavior. The original pipeline seeded all units in the FastAPI store — over a thousand code units — and performed one-hop graph expansion from each. Because `routing.py` and `applications.py` are imported by nearly every other module in the package, they entered the candidate pool from a large fraction of the seed units. Their structure shards (confidence 0.95) and dependency shards (confidence 0.95) ranked at the top of the confidence-only sort. The budget enforcer filled all 8,000 tokens with shards from these two files before any shard from `exception_handlers.py` or `background.py` was reached. The assembled context contained routing table signatures, application factory class hierarchies, and middleware dispatch logic — none of which addresses why background task exceptions escape the exception handler scope. The sharpness of this retrieval is 0.0: every selected shard entered via `fallback_all`.

Post-fix behavior. The fixed pipeline begins with intent resolution. The task string is tokenized into `{fix, exception, handlers, catch, exceptions, raised, background, tasks}` and expanded through the concept map: `exception` expands to `{handler, catch, raise, error, traceback}`, `background` expands to `{task, worker, async, deferred}`. The resulting `SemanticProfile` is dominated by exception- and background-task vocabulary. BM25-lite scoring over the structure shards of the FastAPI store ranks `exception_handlers.py` and `background.py` among the top- N anchors,

because their unit identifiers and structural signatures contain precisely these terms. These units receive retrieval reason `semantic_match` and form the anchor set.

Graph expansion from these anchors proceeds outward by one hop. `routing.py` (degree 31) and `applications.py` (degree 28) are both reachable as neighbors, but both exceed the hub-degree cap of 25 and are skipped unconditionally. The expansion instead pulls in lower-degree neighbors such as `fastapi/responses.py` and `fastapi/concurrency.py`. These receive reason `graph_neighbor`. Modality fusion then fetches behavior, state, and dependency shards for each candidate in the fingerprint order for a `bug_fix` task. The reason-aware budget sort places `semantic_match` shards first, filling the early portion of the 8,000-token budget with `exception_handlers.py` and `background.py` content before any `graph_neighbor` or `modality_expansion` shards are considered. The assembled context is 1,857 tokens. Sharpness is 0.50: half the shards are semantically targeted, half are modality expansions of those targeted units.

The same task under Naive’s independent traversal consumed 72,531 tokens — a 38-fold difference — because Naive also read `fastapi/routing.py` (25,701 tokens) and `fastapi/applications.py` (28,532 tokens) to trace the full exception-handling lifecycle, confirming that the hub problem is not an artifact of the retrieval system but a genuine property of how this codebase is organized. The fixed pipeline avoids loading those files entirely while still surfacing the modules that explain the actual bug.

Table 3 summarizes the changes across all five dimensions affected by the fix.

Dimension	Before	After
Seed strategy	All units in store	Top- <i>N</i> by BM25-lite semantic search
Graph expansion	Unrestricted one-hop BFS	Hub-capped BFS (degree > 25 skipped)
Budget sort key	Confidence only	(reason_rank, −score, −confidence, id)
Behavior shards required	No (quality degrades silently)	No (degrades to structure fallback, logged)
Observability	Token count only	Token count + sharpness + per-shard reason

Table 3: Before and after comparison across the five dimensions affected by the Structural Hub Collapse fix. All changes preserve retrieval determinism.

6 Results

Results are reported across both codebases and all ten tasks. The evaluation measures two quantities: assembled context size in tokens and retrieval sharpness as defined by Equation 1. All runs were conducted in degraded mode, as described in Section 4.2: no behavior shards were generated, and the STEP 2 behavior search fell back to BM25-lite over structure shards.

6.1 Token Efficiency

Table 4 reports the assembled token count for MAR, the file tokens read by Naive, the percentage reduction of MAR relative to Naive, and MAR’s percentage reduction relative to the naive full-context baseline for each task and codebase. Figures 3 and 4 show the per-task comparison on a log scale.

Codebase	Task type	MAR (tok)	Naive (tok)	MAR vs. Naive	MAR vs. baseline
FastAPI	feature_add	4,931	2,306	-113% [†]	99.3%
FastAPI	bug_fix	1,857	72,531	97.4%	99.7%
FastAPI	refactor	6,233	33,674	81.5%	99.1%
FastAPI	explain	3,113	46,473	93.3%	99.5%
FastAPI	onboard	5,114	52,790	90.3%	99.3%
<i>FastAPI avg</i>		<i>4,250</i>	<i>41,555</i>	<i>82.0%</i>	<i>99.4%</i>
Flask	feature_add	7,931	36,600	78.3%	94.1%
Flask	bug_fix	5,626	25,040	77.5%	95.8%
Flask	refactor	7,983	30,022	73.4%	94.1%
Flask	explain	7,026	21,396	67.2%	94.8%
Flask	onboard	5,936	32,324	81.6%	95.6%
<i>Flask avg</i>		<i>6,900</i>	<i>29,076</i>	<i>75.6%</i>	<i>94.9%</i>

Table 4: Token comparison across all 10 tasks. *MAR vs. Naive*: percentage reduction in MAR’s assembled tokens relative to Naive’s file reads. *MAR vs. baseline*: reduction relative to the naive full-context baseline (FastAPI: 689,221 tokens; Flask: 134,479 tokens). [†]T1 outlier: MAR over-retrieved in degraded mode; discussed in Section 6.3.

Across the nine non-outlier tasks, the proposed system assembled an average of 4,661 tokens against Naive’s average of 38,603 tokens, a reduction of 87.9 percent. The bug-fix task on FastAPI produced the largest single reduction: 1,857 tokens versus Naive’s 72,531 tokens, a 38-fold difference, because Naive read `fastapi/routing.py` (25,701 tokens) and `fastapi/applications.py` (28,532 tokens) in full to trace the exception-handling lifecycle, while the proposed system targeted only the two relevant modules. Relative to the naive full-context baselines, the proposed system achieved 99.4 percent reduction on FastAPI and 94.9 percent on Flask: any task in the evaluation could be answered from less than one percent of the total codebase token cost.

6.2 Retrieval Sharpness

Table 5 reports retrieval sharpness, computed using Equation 1, for each task and codebase. Sharpness is a property of MAR’s retrieval only; Naive has no retrieval trace.

Codebase	Task type	Sharpness	Note
FastAPI	feature_add	0.44	outlier; tutorial file not targeted
FastAPI	bug_fix	0.50	—
FastAPI	refactor	0.50	—
FastAPI	explain	1.00	perfect; all shards semantically targeted
FastAPI	onboard	0.41	lowest; DI system spans many modules
<i>FastAPI avg</i>		<i>0.57</i>	
Flask	feature_add	0.70	—
Flask	bug_fix	0.50	—
Flask	refactor	1.00	perfect
Flask	explain	1.00	perfect
Flask	onboard	0.50	—
<i>Flask avg</i>		<i>0.74</i>	

Table 5: Retrieval sharpness per task and codebase. Sharpness = $|T_{\text{targeted}}|/|T|$ (Equation 1). Three tasks reach sharpness 1.0: FastAPI explain, Flask refactor, and Flask explain. The lowest sharpness (0.41) occurs on the FastAPI onboard task, where the dependency injection system is distributed across many modules.

Flask’s average sharpness (0.74) exceeds FastAPI’s (0.57) by 0.17 points. Three tasks reach sharpness 1.0: FastAPI explain and both Flask refactor and explain. These tasks share a property: the modules most relevant to the task are named explicitly enough that structure-based BM25, even in the absence of behavior shards, correctly identifies them as anchors. The explain task on FastAPI targets `fastapi/routing.py`’s serialization logic via `response_model` vocabulary; on Flask, the refactor and explain tasks target `sansio/scaffold.py` and `ctx.py` respectively, both of which have distinctive names and term distributions. The lowest sharpness (0.41) occurs on the FastAPI onboard task, where the dependency injection system is distributed across `params.py`, `dependencies/utils.py`, `dependencies/models.py`, and `routing.py`: no single module carries a dominant portion of the relevant vocabulary, so structure BM25 distributes its budget across a wider set of candidates.

6.3 Task-Level Analysis

The feature_add outlier (FastAPI T1). The task required adding HTTP timing middleware. The correct reference is `docs_src/middleware/tutorial001.py310.py`, a 47-line tutorial file containing fewer than 200 tokens. Naive located this file directly by reading three tutorial directories, consuming 2,306 tokens in total. In degraded mode, the proposed system’s BM25 structure search ranked exception classes and test files above the tutorial because they scored higher on the word *middleware* at the structural level — the tutorial file’s unit identifier and signature are sparse. MAR assembled 4,931 tokens, selecting the wrong units. This task represents the clearest empirical demonstration of what behavior shards are designed to fix: an intent summary of the tutorial would read *demonstrates how to add HTTP middleware using the @app.middleware decorator*, a direct semantic match that structure-only BM25 cannot replicate.

Sharpness 1.0 tasks. The three tasks achieving perfect sharpness produced among the strongest token reductions: FastAPI explain at 93.3 percent and Flask explain at 67.2 percent against Naive’s file reads. In all three cases, every selected shard carried a `semantic_match` or `anchor_file` reason; no shard entered via the fallback path. The correlation between sharpness and retrieval precision

is direct: when every shard in the budget is intent-guided, the assembled context contains exactly what the task requires and nothing else.

Hub confirmation from Naive. Naive read `fastapi/routing.py` in three of five FastAPI tasks and `fastapi/applications.py` in two of five, despite these files being relevant only to the refactor task. On Flask, `src/flask/app.py` appeared in three of five traversals. This pattern, observed independently of the retrieval system, confirms that the hub-collapse tendency described in Section 5 is a structural property of framework codebases rather than an artifact of the proposed system’s design.

7 Discussion

7.1 Context as Cost, Not Capacity

The framing of this paper treats context size not as a resource to be maximized but as a cost to be minimized. The token budget consumed by source code is directly subtracted from the space available for reasoning, tool use, and multi-step generation. This framing has a practical consequence that raw reduction percentages do not capture: a 97 percent reduction in context size does not just save tokens, it changes what is possible. A task whose relevant code fills 50,000 tokens cannot be addressed at all within a 32,000-token context window, and is degraded even within a 200,000-token window when intermediate reasoning and tool interaction consume the bulk of the effective capacity. The results show that 9 of 10 tasks in this evaluation were answerable from under 8,000 tokens of assembled context, which represents less than 4 percent of the total capacity of any current frontier model. The remaining 96 percent of the context window is available for chain-of-thought reasoning, tool calls, and response generation — the operations that produce actual task output.

7.2 Degraded Mode and the Role of Behavior Shards

Every result reported in Section 6 was produced without behavior shards. The `ANTHROPIC_API_KEY` was not set at ingestion time, and the STEP 2 behavior search fell back to BM25-lite over structure shards. This is a material constraint on the results, not a minor technical footnote. Behavior shards carry intent summaries generated by a language model: they encode what a unit *does*, not just what it *is called*. Structure shards encode only syntactic shape: signatures, class hierarchies, and decorator lists. BM25-lite over structure shards can identify a module named `exception_handlers.py` when the task mentions exceptions, but it cannot identify a 47-line tutorial file whose intent matches the task and whose unit name is `docs_src.middleware.tutorial001.py310`.

The `feature_add` outlier on FastAPI (T1) quantifies this limitation precisely. With behavior shards, the intent summary of the tutorial would have been a near-exact match for the task query, and the retrieval anchor set would have contained the correct file. Without them, structure BM25 selected exception classes and test files that scored higher on structural features. The proposed system over-retrieved by 2,625 tokens relative to Naive’s targeted traversal. This is not a failure of the four-stage pipeline; it is a demonstration of what the pipeline does with insufficient signal. Degraded mode is the realistic deployment floor for teams that have not run a full ingestion pass, and the results show it is still useful: 9 of 10 tasks achieved significant token reduction. But the

gap between the T1 outlier and the T4 explain task (sharpness 0.44 vs 1.00) marks the range of improvement that behavior shards are expected to close.

7.3 Sharpness as a Retrieval Quality Proxy

Retrieval sharpness correlates with retrieval precision in every case where ground truth can be assessed. The three tasks reaching sharpness 1.0 all produced context payloads where every selected shard was confirmed to be relevant to the task. The task with the lowest sharpness (FastAPI onboard, 0.41) produced a payload whose onboarding overview was architecturally correct but less specific than Naive’s answer, which traced the dependency injection resolution path through multiple files at the implementation level. Sharpness is not a direct measure of answer quality — a low-sharpness retrieval can still produce a correct answer — but it is a reliable signal of whether the retrieval mechanism operated as intended.

A low sharpness score carries a specific diagnostic meaning: more than half of the assembled shards entered via `modality_expansion` or `fallback_all` rather than via semantic matching or graph expansion from a matched anchor. This indicates that the semantic anchor set was sparse or poorly matched to the task, and the budget was filled by topological proximity rather than intent. In the context of degraded-mode retrieval, this is expected: structure BM25 is a weaker signal than behavior BM25, and some tasks produce semantic profiles that do not match any structure shard distinctively. In full-ingestion mode, where behavior shards carry intent summaries, the same tasks would produce denser, more precise anchor sets, and sharpness would increase correspondingly.

The codebase-size effect on sharpness (Flask 0.74, FastAPI 0.57) supports this interpretation. With fewer code units competing for the top- N anchor slots, BM25 term overlap is more distinctive: a term that appears in 3 of 83 modules carries more signal than the same term appearing in 3 of 1,118. As codebase size grows, structure-only BM25 degrades more rapidly, and behavior shards become proportionally more valuable. This suggests that the proposed system’s advantage over degraded mode is largest precisely where the context window problem is most acute: in large codebases.

7.4 Hub Collapse as a Structural Codebase Property

The results of Naive’s independent traversal confirm that Structural Hub Collapse is not an artifact of the proposed system’s graph expansion but a genuine property of how large Python frameworks are organized. An unassisted agent, given no retrieval system at all and only file-reading tools, gravitates toward the same hub modules: `routing.py` appeared in three of five FastAPI traversals, `applications.py` in two of five, and `app.py` in three of five Flask traversals. These files are read not because they contain the answer to the task but because they are the natural entry point for a reader trying to understand any aspect of the framework. They contain cross-references, dispatch logic, and wiring code that gives the impression of relevance to almost any task.

This has an implication for the hub-degree cap design: the cap is correctly applied to graph expansion neighbors rather than to semantic anchors. A hub module can be genuinely relevant to a task — a refactor task targeting the routing table is an example. The cap prevents hub modules from being pulled in passively as collateral of graph traversal from an unrelated anchor. It does not prevent them from being targeted directly when the task query semantically matches their content. In practice, across all 10 tasks in this evaluation, no hub module was the primary semantic

anchor for any task, which is consistent with the intended design: hubs provide infrastructure, not answers.

7.5 Limitations

The evaluation reported here operates entirely in degraded mode. All reported token reductions and sharpness scores reflect the system without behavior shards, which means the results underestimate the system’s capability in full-ingestion deployment. A direct comparison between degraded-mode and full-ingestion retrieval on the same task set would quantify this gap precisely, but requires running the ingestion pipeline with a live API key at the scale of FastAPI, which was not done for this evaluation.

Task output quality was assessed qualitatively by inspection rather than by an automated judge. For most tasks, both agents produced functionally equivalent answers at the architectural level, which is consistent with the expectation that the relevant knowledge is largely present in both context payloads. For the T1 outlier, Naive produced a more precisely targeted answer; for T2 bug fix, MAR’s answer was architecturally correct but less specific than Naive’s line-level analysis. These qualitative differences are noted in Section 6.3 and are consistent with the sharpness scores but are not scored numerically in this evaluation.

The evaluation covers two Python web frameworks. Python is the only language supported in the current ingestion pipeline, and both codebases share the general structural pattern of framework repositories (high-degree hub modules, a layered architecture, a large documentation source tree). The generalization of these results to compiled-language codebases, domain-specific repositories, or codebases with a flat module structure is not assessed here and constitutes a direction for future work.

8 Conclusion & Future Work

This paper presents a modality-aware retrieval architecture for LLM-based code agents that addresses a fundamental inefficiency in how language models consume codebase context. Rather than loading files, the proposed system decomposes every code unit into five knowledge modalities—structure, behavior, dependencies, state, and history—and retrieves only the modality-shard combinations required for the task at hand. The experimental evaluation across FastAPI and Flask demonstrates a mean token reduction of 82.0% and 75.6% respectively relative to a naive full-traversal baseline, both results obtained in degraded mode without behavior shards.

A key contribution of this work is the identification and resolution of *Structural Hub Collapse*, a failure mode in which topology-biased retrieval systematically excludes task-relevant code in favor of high-degree infrastructure modules. The four-stage semantic pipeline—intent resolution, BM25-lite behavior search, hub-constrained graph expansion, and reason-aware budget enforcement—addresses this failure without relying on neural retrieval or LLM calls at query time. The accompanying *retrieval sharpness* metric provides a cheap, deterministic proxy for retrieval quality that requires no ground-truth annotation and is computable from the retrieval trace alone.

Future Work

Multi-language support. The current ingestion pipeline is Python-specific: the decomposer relies on `ast` (stdlib), `git log`, and `tiktoken` for tokenization. Extending MAR to a universal code intelligence tool requires replacing the Python-specific AST pass with a language-agnostic parser. The `tree-sitter` library already provides grammars for JavaScript, TypeScript, Go, Rust, Java, C, C++, Ruby, and dozens of additional languages. A unified `decomposer` interface over `tree-sitter` would allow structure, dependency, and state parsers to operate on any supported language with minimal per-language configuration. History and behavior shards are already language-agnostic: `git log` parses commits regardless of language, and the LLM call in the behavior parser requires only that the source fragment be readable. A cross-language ingestion layer would make MAR directly applicable to polyglot monorepos and compiled-language systems, the two classes of production codebase where context costs are highest.

Improved LLM support for behavior shards. Behavior shards are the highest-value modality for retrieval precision—they carry semantic intent rather than syntactic form—but their generation requires one LLM call per code unit, making full ingestion of large codebases expensive at current API pricing. Several improvements are planned. First, structured prompting with constrained output schemas (JSON-mode or tool-use) would reduce behavior shard token length while increasing parsability. Second, batched ingestion pipelines that coalesce units from the same file into a single prompt can reduce call count by 3–5× for typical module sizes. Third, integrating open-weight models (e.g., Code Llama, DeepSeek-Coder, Qwen-Coder) as a behavior shard backend would allow offline, zero-cost ingestion on developer hardware. Finally, a cost-aware ingestion strategy that prioritizes behavior shard generation for high-centrality units (those most likely to be retrieved) would give near-full retrieval benefit at a fraction of the per-unit cost.

Full-ingestion evaluation and ablation. All results in this paper are from degraded-mode ingestion (no behavior shards). The complete four-modality system with behavior shards is expected to raise sharpness substantially, particularly for tasks whose intent matches no structure-shard keyword but is captured in a behavior summary. A planned 4-mode ablation—structure-only, behavior-only, hybrid, and full—over the same FastAPI and Flask task sets will quantify the independent contribution of each modality and validate the sharpness metric as a retrieval quality predictor.

Incremental and real-time re-ingestion. The current pipeline is batch-mode: the entire codebase must be re-ingested after any change. In practice, developers modify a small fraction of files per commit. An incremental ingestion layer, triggered by `git diff` output, would re-parse and re-store only the affected code units and update the networkx graph in place. This would make MAR viable as a persistent background process in an IDE integration, maintaining a live shard store that is always in sync with the working tree.

Automated retrieval quality evaluation. Retrieval sharpness measures intent-alignment without reference to ground truth, but it cannot detect cases where semantically relevant code is missed entirely (false negatives). A planned evaluation harness will use a second LLM call—blind to the retrieval method—to score assembled context payloads on task-specific criteria: correctness, completeness, and absence of irrelevant content. This blind evaluation, applied uniformly to MAR and Naive outputs, will provide the first direct comparison of output quality rather than token efficiency alone.

Multi-repository retrieval. Large engineering organizations maintain dozens of interdependent repositories. A cross-repo context assembly layer would extend the networkx graph across repository boundaries, treating package imports as IMPORTS edges that traverse into separately-ingested shards. This is architecturally straightforward given the existing shard store design but requires a shared unit namespace across repositories and a stable cross-repo identifier scheme.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*. [online] <https://arxiv.org/abs/2005.11401>
- [2] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. 2023. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12, 157–173. [online] <https://arxiv.org/abs/2307.03172>
- [3] T. J. Teyler and P. DiScenna. 1986. The Hippocampal Memory Indexing Theory. *Behavioral Neuroscience*, 100(2), 147–154.
- [4] J. L. McClelland, B. L. McNaughton, and R. C. O’Reilly. 1995. Why There Are Complementary Learning Systems in the Hippocampus and Neocortex: Insights from the Successes and Failures of Connectionist Models of Learning and Memory. *Psychological Review*, 102(3), 419–457. [online] <https://doi.org/10.1037/0033-295X.102.3.419>
- [5] I. Beltagy, M. E. Peters, and A. Cohan. 2020. Longformer: The Long-Document Transformer. [online] <https://arxiv.org/abs/2004.05150>
- [6] H. Husain, H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt. 2019. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. [online] <https://arxiv.org/abs/1909.09436>
- [7] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*. [online] <https://arxiv.org/abs/2405.15793>
- [8] GitHub. 2023. GitHub Copilot. [online] <https://github.com/features/copilot>
- [9] Anysphere. 2024. Cursor. [online] <https://cursor.com>
- [10] Codeium. 2024. Windsurf. [online] <https://windsurf.com>